Student notes — Session 12 · String manipulation and conversions

Slides: [Class_12_String_manipulation_and_conversions.pdf](../slides/Class_12_String_manipulation_and_conversions.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Quiz 3: 25 minutes

This slide was on the screen during Quiz 3. Your marked paper comes back next week, on Blackboard.

Quiz 3 covered classes 7 to 10. Quiz 4 is on **Friday 23 October** and covers classes 11 and 12: characters and strings, and today's class.

Slide 2 — String manipulation and conversions

The programs of this class are on Blackboard: Week 6 module, *Class 12 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Today: the string functions of `<string.h>`, two things written by hand, and how to turn text into numbers and back.

Slide 3 — What Tuesday gave us

What you need from Tuesday:

- A string is an array of `char` that ends with the end mark `'\0'`.
- `char city[20] = "Baku";` keeps room for 19 letters and the end mark.
- `%s` prints a string. `scanf("%19s", city)` reads one word. No `&` for a string.
- A loop can walk along a string until it meets `'\0'`.
- `'7' - '0'` is the number `7`.

Slide 4 — Recap: what is on the screen?

Try to answer before you look at the next slide.

Slide 5 — Recap: what is on the screen?

The screen shows `length = 4`. The loop looks at B, a, k, u and stops at `'\0'`. The end mark is not counted. The file is `01_recap_length.c`.

Slide 6 — Today's roadmap

Five parts:

1. **Length and copy**: `strlen`, `strcpy`.
2. **Join**: `strcat`.
3. **Compare**: `strcmp`.
4. **By hand**: reverse a word, test a palindrome.
5. **Text and numbers**: `"2026"` into `2026`, and back.

Slide 7 — Length and copy

Part 1: the string toolbox.

Slide 8 — A toolbox for strings

Put this line at the top of your program:

```
#include <string.h>
```

It gives you four functions:

- `strlen(s)`: how many characters `s` has.
- `strcpy(to, from)`: copy a string into an array.
- `strcat(to, more)`: add a string to the end of another.
- `strcmp(a, b)`: compare two strings.

Each of them is a loop to `'\0'`, like the loops you wrote on Tuesday.

Slide 9 — strlen: the length

The file `02_strlen.c`. It prints:

```
strlen: 4
sizeof: 20
```

`strlen(city)` counts the characters before `'\0'`: B, a, k, u, so 4. `sizeof(city)` counts all the boxes of the array, used or not: 20.

`strlen` gives a number of a special size type. `(int)` turns it into an `int`, so `%d` can print it. Write it that way.

Slide 10 — strlen is not sizeof

The array `city` box by box:

box	0	1	2	3	4	5 to 19
`city`	B	a	k	u	`\0`	empty

`strlen(city)` is 4. `sizeof(city)` is 20.

Slide 11 — A loop up to strlen

The file `03_strlen_loop.c`. `n = strlen(word)` is 4, so the loop runs for `i` = 0, 1, 2, 3 and prints each letter with a space:

```
B a k u
```

Slide 12 — Careful: give the array a new name

Try to answer before you look at the next slide: can you write `name = "Leyla";`?

Slide 13 — No: = does not work for arrays

The file `04_bug_assign.c`. It does **not** compile. gcc says:

```
error: assignment to expression with array type
```

You cannot give a whole array a new value with `=`. You may write `= "Leyla"` only in the line that makes the array:

```
char name[20] = "Leyla";   // allowed: this line makes the array
name = "Leyla";            // not allowed: error
```

Slide 14 — strcpy: copy into the array

The file `05_strcpy.c`. To put a new string into an array, use `strcpy`:

```
strcpy(name, "Leyla");
```

It copies L, e, y, l, a and the `'\0'` into `name`. A second `strcpy(name, "Murad");` writes over it. The program prints `Leyla` and then `Murad`.

The order is `strcpy(to, from)`, the same order as `to = from`.

Slide 15 — Join

Part 2: join two strings.

Slide 16 — strcat: add to the end

The file `06_strcat.c`. `strcat(to, more)` adds `more` to the end of `to`:

```
char full[30] = "Ada";
strcat(full, " ");
strcat(full, "Lovelace");
```

It prints `Ada Lovelace` and `12 characters`. `full` has 30 boxes, so there is room for everything.

Slide 17 — full, line by line

after the line	`full`	`strlen`
`char full[30] = "Ada";`	`Ada`	3
`strcat(full, " ");`	`Ada`	4
`strcat(full, "Lovelace");`	`Ada Lovelace`	12

Each `strcat` writes over the old `'\0'` and puts a new one after the last letter.

Slide 18 — Careful: is there room?

Try to answer before you look at the next slide. `small` has 6 boxes. What happens when you add `" there!"`?

Slide 19 — gcc warns: it overflows

The file `07_bug_strcat_room.c`. gcc compiles it, but warns:

```
warning: '__builtin_memcpy' writing 8 bytes into a region of size 6 overflows the
destination
```

The result, `"Hi there!"`, has 9 characters. With `'\0'` it needs **10** boxes. `small` has only 6.

Slide 20 — And yet it prints

When you run it, it prints `Hi there!`. That does **not** mean it is right. `strcat` wrote past the end of `small`, into memory that belongs to something else. In a bigger program that breaks other variables, and the bug is very hard to find. A wrong program can look right.

Slide 21 — The rule for strcpy and strcat

The array must have room for the result **plus one** box for `'\0'`.

- `"Hi there!"`: 9 characters + 1 = 10 boxes.
- C does not check this for you. You count.
- When in doubt, make the array bigger: `char full[100];`

Slide 22 — Compare

Part 3: compare two strings.

Slide 23 — Predict: you type yes

Try to answer before you look at the next slide. You type `yes`. What does the program print?

Slide 24 — It says Stopping

The file `08_bug_compare_eq.c`. You type `yes` and it prints:

```
Continue? yes
Stopping
```

The `if (answer == "yes")` was false, even though you typed exactly `yes`.

Slide 25 — gcc saw it

When you compile with `-Wall`, gcc warns:

```
warning: comparison with string literal results in unspecified behavior
```

Read your warnings. This one says the program is wrong.

Slide 26 — == does not compare letters

For strings, `==` does not compare the letters. It asks whether the two are **the same array**. `answer` is one array and `"yes"` is another, so `==` is always false.

To compare the letters, write:

```
if (strcmp(answer, "yes") == 0)
```

Slide 27 — strcmp: three answers

The file `09_strcmp.c` prints, on the course's gcc:

```
-1
1
0
-1
```

for `strcmp("apple", "banana")`, `strcmp("banana", "apple")`, `strcmp("apple", "apple")` and `strcmp("Zoo", "apple")`. Another computer may print other numbers. Only the **sign** is the same everywhere.

Slide 28 — What strcmp(a, b) says

result	meaning	example
negative	`a` comes first	`strcmp("apple", "banana")`
0	equal	`strcmp("apple", "apple")`
positive	`a` comes later	`strcmp("banana", "apple")`

Capital letters come before small letters, because their ASCII codes are smaller: `'Z'` is 90 and `'a'` is 97. So `"Zoo"` comes before `"apple"`.

Slide 29 — A password check

The file `10_password.c`. The test for "equal" is `strcmp(...) == 0`:

```
Password: ada2026
Welcome

Password: ADA2026
Wrong password
```

`ADA2026` is wrong because capital letters are different characters. Careful: `if (strcmp(a, b))` without `== 0` is true when the strings are **different**.

Slide 30 — Which name comes first?

The file `11_first_name.c`. It reads two names and uses the sign of `strcmp`: `strcmp(a, b) < 0` means `a` comes first.

```
Two names: Murad Aysel
First: Aysel
```

Slide 31 — By hand

Part 4: two jobs that `string.h` does not do. You write them yourself.

Slide 32 — Reverse: swap from both ends

The file `12_reverse.c`. The function `reverse`:

- `i` starts at the first letter, `j = strlen(s) - 1` at the last.
- Swap `s[i]` and `s[j]` with `temp`, the swap from class 10.
- `i++` and `j--`: one step inward. Stop when `i < j` is false.

There is no size parameter: a string has its own end mark, so `strlen` finds the length. The array is not copied, so `main`'s word changes.

Slide 33 — Reverse "hello", step by step

`i`	`j`	swap	`word`
-	-		`hello`
0	4	h, o	`oellh`

Slide 34 — Reverse "hello", step by step

i	j	swap	word
1	3	e, l	olleh
2	2	stop	olleh

The middle letter stays where it is. The program prints `olleh`.

Slide 35 — A palindrome reads the same both ways

The file `13_palindrome.c`. A palindrome reads the same from both ends, like `level`. The function compares `s[i]` with its partner from the end, `s[n - 1 - i]`, for the first half of the word. As soon as one pair differs it returns 0. If every pair matched, it returns 1.

Slide 36 — level, kayak, hello, abba

```
level: yes
kayak: yes
hello: no
abba: yes
```

`check("level")` sends a string in quotes straight to a function. Note that `"Level"` with a capital L is **not** a palindrome for this function: `'L'` and `'l'` are different characters.

Slide 37 — Text and numbers

Part 5: text and numbers.

Slide 38 — "2026" is not 2026

- `"2026"` is a string: the characters `'2'`, `'0'`, `'2'`, `'6'`, then `'\0'`.
- `2026` is one `int`. You can add to it, compare it, multiply it.

Input often arrives as text, so you need to convert.

Slide 39 — Text to number, by hand

The file `14_digits_to_int.c`:

```
n = n * 10 + (s[i] - '0');
```

`s[i] - '0'` turns a digit character into its number (`'6' - '0'` is 6). `n * 10` moves what you have one place to the left, and the new digit goes into the ones place.

Slide 40 — "2026" becomes 2026

i	s[i]	digit	n
0	'2'	2	0 * 10 + 2 = 2
1	'0'	0	2 * 10 + 0 = 20

Slide 41 — "2026" becomes 2026

i	s[i]	digit	n
2	'2'	2	20 * 10 + 2 = 202

i	s[i]	digit	n
3	'6'	6	202 * 10 + 6 = 2026

Then `s[4]` is `'\0'` and the loop stops.

Slide 42 — It is a real number now

The program prints:

```
n = 2026
n + 1 = 2027
```

You can add 1 to it, so it is a real number.

Slide 43 — atoi does the same job

The file `15_atoi.c`. `atoi` turns text into an `int`. It is in `<stdlib.h>`, not in `<string.h>`.

```
atoi("2026") + 1    gives 2027
atoi("-45")         gives -45
atoi("12abc")       gives 12   (it stops at 'a')
atoi("abc")         gives 0    (no number at the start)
```

Careful: `atoi` also gives 0 for text that is not a number, so 0 can mean "zero" or "not a number".

Slide 44 — Number to text, by hand

The file `16_int_to_string.c`. The other way:

- `n % 10` is the last digit: 472 gives 2.
- `'0' + 2` is the character `'2'`.
- `n / 10` drops the last digit: 472 becomes 47.

Repeat while `n > 0`, then put `'\0'` at the end. It prints `backwards: 274`.

Slide 45 — 472, digit by digit

n	n % 10	s	n / 10
472	2	"2"	47
47	7	"27"	4
4	4	"274"	0

The last digit comes out first, so the string is backwards.

Slide 46 — Then reverse it

The same swap as in `reverse` turns `"274"` into `"472"`. The program prints `text: 472`. (Try: what does the program do with `n = 0`? The loop never runs. Fix it.)

Slide 47 — sprintf: printf into a string

The file `17_sprintf.c`. `sprintf` works like `printf`, but it writes into an array instead of onto the screen:

```
sprintf(s, "%d", 472);                                  // s is "472"
snprintf(msg, sizeof(msg), "%s scored %d", "Aysel", 95); // msg is "Aysel scored 95"
```

`snprintf` gets the size of the array as its second argument and never writes past it. Prefer it.

Slide 48 — Text and numbers: two ways each

job	by hand	with a function
text to `int`	`n = n * 10 + (s[i] - '0')`	`atoi(s)` (`<stdlib.h>`)
`int` to text	`n % 10`, `n / 10`, reverse	`sprintf(s, "%d", n)`

Know both. A quiz can ask you to do it by hand.

Slide 49 — Your turn

Part 6: two short tasks. Write them on paper first, then run them.

Slide 50 — Your turn: digit_sum

Write `int digit_sum(char s[])`. It adds up the digits of a string: `digit_sum("2026")` is 2 + 0 + 2 + 6 = 10. Try before you look at the next slide.

Slide 51 — digit_sum: one answer

The file `18_task_digit_sum.c`:

```
int digit_sum(char s[]) {
    int sum = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        sum += s[i] - '0';
    }
    return sum;
}
```

`digit_sum("2026")` is 10 and `digit_sum("999")` is 27.

Slide 52 — Your turn: an e-mail address

Read a user name, like `aysel`, and print `aysel@ada.edu.az`. Use `strcpy` and `strcat`. How big must the array be? Try before you look at the next slide.

Slide 53 — E-mail: one answer

The file `19_task_email.c`:

```
strcpy(email, user);
strcat(email, "@ada.edu.az");


User name: aysel
aysel@ada.edu.az
```

`email` has 40 boxes: the name (up to 19 letters), the 11 characters of `@ada.edu.az`, and one for `'\0'`.

Slide 54 — Today in one slide

- `strlen` counts the characters before `'\0'`. It is not `sizeof`.
- Put a string into an array with `strcpy`, never with `=`.

- `strcat` adds to the end. Count the room: the result plus one.
- Two strings are equal when `strcmp(a, b) == 0`. Never use `==`.
- Text to number: `n * 10 + digit` or `atoi`. Number to text: `% 10` and `/ 10`, or `sprintf`.

Slide 55 — Before Tuesday

- **Knowledge check 12** is on Blackboard. It is about today's slides.
- **Quiz 4 is on Friday 23 October.** It covers classes 11 and 12: characters and strings.
- **Project 2** is announced this week, on Blackboard.
- **Midterm**: in the week of 26 October. It covers classes 1 to 14. The date and the room will be announced.
- Today's code is in the Week 6 module: **Class 12 code**.

Next class: two-dimensional arrays, tables of numbers.